CZECH TECHNICAL UNIVERSITY IN PRAGUE
Faculty of Electrical Engineering
Department of Control Engineering

FOC Firmware for Integrated BLDC Driver

Bachelor's thesis

Mathias Palme

Supervisor
Ing. Krištof Pučejdl

2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Palme** Jméno: **Mathias** Osobní číslo: **516266**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Otevřená informatika**

Specializace: **Základy umělé inteligence a počítačových věd**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

FOC firmware pro integrovaný BLDC driver

Název bakalářské práce anglicky:

FOC Firmware for Integrated BLDC Driver

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Krištof Pučejdl katedra řídicí techniky FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **26.01.2026** Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Palme Mathias

Podpis studenta

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Palme** Jméno: **Mathias** Osobní číslo: **516266**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Otevřená informatika**

Specializace: **Základy umělé inteligence a počítačových věd**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

FOC firmware pro integrovaný BLDC driver

Název bakalářské práce anglicky:

FOC Firmware for Integrated BLDC Driver

Pokyny pro vypracování:

The goal of this thesis is developing and testing embedded software that implements Field Oriented Control (FOC) for Brushless DC (BLDC) motors on an existing integrated three-phase motor driver with an STM-32 type microcontroller.
- Familiarize yourself with principles of FOC algorithm and review the existing implementations that use similar electronic components.
- Familiarize yourself with the specific uFOC hardware components their function, and the overall board layout and schematic.
- Develop and implement a firmware allowing the uFOC driver to control a BLDC motor using the FOC principle. Implement control modes for commanding output Torque, Velocity, and Position, and communication protocol via CAN bus, allowing setting, commanding and reading driver status from different devices connected on the bus.
- Create a documentation for the firmware on the existing GitHub repository.
- Test the firmware and demonstrate the control via simple benchmark application (for example a simple haptic device).

Seznam doporučené literatury:

[1] R. Gabriel, W. Leonhard and C. J. Nordby, „Field-Oriented Control of a Standard AC Motor Using Microprocessors,“ in IEEE Transactions on Industry Applications, 1980, doi: 10.1109/TIA.1980.4503770.
[2] A. Skuric et al., „SimpleFOC: A Field Oriented Control (FOC) Library for Controlling Brushless Direct Current (BLDC) and Stepper Motors,“ in Journal of Open Source Software, 2022, doi: 10.21105/joss.04232.
[3] R. C. Prayogo, A. Triwiyatno and M. A. Riyadi, „Field Oriented Control Implementation on BLDC Motor Controller with PI and SVPWM using STM32F103C8T6,“ in Journal of Physics: Conference Series (Vol. 2622). Institute of Physics, 2023, doi:10.1088/1742-6596/2622/1/012025

Declaration

I hereby declare that this thesis is my original work and it has been written by me in its entirety. I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Mathias Palme
May 2026

Acknowledgments

Acknowledgment section need not to be too strict. Try to express your personality, sense of humor and so on :).

Abstract

The goal of this thesis is developing and testing embedded software that implements Field Oriented Control (FOC) for Brushless DC (BLDC) motors on an existing integrated three-phase motor driver called uFOC (micro FOC) with an STM-32 type microcontroller developed by Bc. Šimon Pecháček and Ing. Krištof Pučejdl at the Department of Control Engineering, Faculty of Electrical Engineering, Czech Technical University in Prague. The firmware implements FOC control algorithm to control torque. On top of torque control a PI velocity and PID position control loop is implemented. Aditionally communication via CAN bus is implemented with daisy chaining of multiple drivers, to allow indepemendent control and sensor readings from each driver via a signle CAN bus line.

Keywords: FOC, BLDC, embedded software, motor control.

Abstrakt

Česká verze abstraktu (anotace) s klíčovými slovy na konci. Doplnit českou verzi...

Klíčová slova: antikolizní systém, digitální mapa, kalmanův filtr, V2X-komunikace.

Contents

1 | Introduction

BLDC motors are increasingly more popular in varous applications, such as robotics, drones, electric vehicles and many more, due to their high efficiency, ability to produce large torque at low speeds and high position accuracy. Unlike brushed DC motors, BLDC motors need specialized driver and control to achieve commutation.

While commercial BLDC drivers are available on todays market, such as ODrive Micro, moteus-c1 (add reference), they are often expensive in range of 70-90 USD per driver. Therefore, the Department of Control Engineering at the Czech Technical University in Prague has developed an integrated three-phase motor driver called uFOC (micro FOC) with an STM-32 type microcontroller, which is a low-cost alternative to commercial drivers at BOM price of approximately 20 USD.

To achive an efficient commutation and generate torque control, the driver needs to implement a closed loop control. The most common control method for BLDC motors is Field Oriented Control (FOC), which allows for precise control of the motor's torque and speed by controlling the current in the stator windings.

Goal of this thesis is to develop embedded firmware that implements FOC current control, speed and position control loops and communication via CAN bus for the uFOC driver. Additionally a Python client side example for CAN communication with the driver is provided.

2 | Introduction to FOC - work name

2.1 FOC overview

Field Oriented Control is a closed loop control method for controlling torque of BLDC motors based on Clarke-Park Transformation. Standard BLDC motor is aquipped with three stator windings, which are supplied with three phase currents. To achive commutation and generate torque, propper voltage has to be provided to the stator windings by reading the phase currents (i_a, i_b, i_c). Three phase currents reference frames can be decomposed into two orthogonal torque and flux reference frames using Clarke transformation, resulting in two currents (i_α, i_β) in a stationary reference frame.

Park transformation is then applied to transform the currents from a stationary reference frame to a rotating reference frame, resulting in two DC currents (i_d, i_q), where i_d is a flux current and i_q is a torque current.

To control i_d and i_q currents, two independent proportional-integral (PI) controllers are used to update applied voltage to the stator windings based on the error between the reference and the measured currents. In this implementation only the i_q current reference is changed to control the torque, while the i_d current reference is always set to zero to maximize the torque per ampere.

After getting new voltage references from the PI controllers, they need to be transformed back to three phase voltages using inverse Park and inverse Clarke transformations, which can be then applied to the stator windings.

However applying the voltages directly from the inverse Park-Clarke transformations will achieve only aproximatelly 84 of the maximum voltage difference between the stator windings, which is not optimal. To maximize the voltage difference between the stator windings, Space Vector Modulation (SVPWM) can be applied to the voltage references.

Now that we can control the torque of the motor by controlling the i_q current reference, a velocity control loop can be implemented on top of the torque control by using a PI controller to update the i_q current reference based on the velocity error.

Aditionally, a position control loop can be implemented on top of the velocity control loop by using a PID controller to update the velocity reference based on the position error.

add diagram of the control loops

2.2 Clarke transformation

Clarke transformation is used to transform three phase currents (i_a, i_b, i_c) into two orthogonal currents (i_α, i_β) in a stationary reference frame, where i_α is torque generating current and i_β is flux producing current.

Clarke transformation can be represented as a matrix multiplication of the three phase currents with the following transformation matrix:

$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix} \tag{2.1}$$

The scaling factor $\frac{2}{3}$ ensures amplitude invariance between the three-phase and i_α i_β representations.

Inverse Clarke transformation in context of FOC is used to transform two orthogonal voltages (v_α, v_β) in a stationary reference frame back to three phase voltages (v_a, v_b, v_c) that can be applied to the stator windings.

$$\begin{bmatrix} v_a \\ v_b \\ v_c \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} v_\alpha \\ v_\beta \end{bmatrix} \tag{2.2}$$

2.3 Park transformation

Park transformation is used to transform two orthogonal currents (i_α, i_β) in a stationary reference frame from the stator point of view into two DC currents (i_d, i_q) in a rotating reference frame from a rotor point of view, where i_d is a flux current and i_q is a torque current. To perfrom Park transformation, the electrical angle θ_e must to be calculated based on the rotor position, which can be obtained from the encoder reading.

Park transformation can be represented as a matrix multiplication of the two orthogonal currents (i_α, i_β) with the following transformation matrix:

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \begin{bmatrix} \cos(\theta_e) & \sin(\theta_e) \\ \sin(-\theta_e) & \cos(\theta_e) \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} \tag{2.3}$$

In context of FOC, inverse Park transformation is used to transform two orthogonal DC voltages (v_d, v_q), which are the outputs of the PI controllers, from a rotating reference frame back to two orthogonal voltages (v_α, v_β) in a stationary reference frame.

Inverse park transformation can be represented as a matrix multiplication of the two orthogonal

DC voltages (v_d, v_q) with the following transformation matrix:

$$\begin{bmatrix} v_\alpha \\ v_\beta \end{bmatrix} = \begin{bmatrix} \cos(\theta_e) & -\sin(\theta_e) \\ \sin(\theta_e) & \cos(\theta_e) \end{bmatrix} \begin{bmatrix} v_d \\ v_q \end{bmatrix} \tag{2.4}$$

2.4 PI regulator

Anti-windup

2.5 Space vector modulation

Space vector modulation is used to maximize the voltage difference between the stator windings.

2.6 Velocity and position control loops

2.7 Electrical angle calculation

3 | Firmware implementation

3.1 Used technologies and development tools

STM32CubeMX was used to simplify the initial project setup, while the HAL library was employed to facilitate peripheral configuration and management. The firmware was implemented in the C programming language. CAN communication testing was performed using a Makerbase CANable V2.0 USB interface together with a Python based client script built on the pycan library. * Also compilation should be mantioned *

Arduino IDE was used as a serial monitor for debugging purposes.

3.2 Firmware topology

The firmware is structured into reusable modules located in the `uFOC_lib` directory. Peripheral initialization and high-level application logic are implemented in `src/main.c`.

uFOC_lib contains the following modules:

- `driver` - handles DRV8316CRRGFR driver configuration and control, ADC current sensing and PWM generation.
- `encoder` - handles MT6835GT encoder SPI configuration, positional data reading, angular velocity calculation and signal filtering.
- `foc` - implements all necessary transformations, to perform FOC control as well as goniometric functions optimizations and SVPWM calculations.
- `config` - is used to store and control global configuration parameters of the firmware. This module implements an abstraction layer to simplify the access to the configuration parameters and ensure that they are updated correctly.
- `pi_controller` - implements PI and PID controllers with anti-windup and output saturation.
- `communication` - manages CAN communication, including message parsing and sending.

The current control loop, together with current sampling and encoder reading, is executed in the ADC injected conversion complete interrupt, which is triggered by the `TIM1_CC4` event at a frequency of 5 kHz. This ensures precise alignment between current measurement and the PWM cycle.

Higher-level control loops, namely velocity and position control, are executed in a separate timer interrupt (`TIM6`) at a lower frequency of 1125 Hz, reflecting the cascaded structure of the control system.

This separation of control loops follows common practice in motor control systems, where fast inner loops regulate currents and slower outer loops regulate mechanical quantities.

3.3 MCU configuration

The uFOC driver uses an STM32F3xx microcontroller as the main processing unit. The MCU is responsible for real-time motor control, peripheral coordination, and communication with external devices. The configuration of the microcontroller is designed to meet the deterministic timing and low latency requirements of FOC.

3.3.1 System clock

The system clock is derived from the internal high-speed oscillator (HSI) operating at 8 MHz. This signal is multiplied using a phase-locked loop (PLL) with a multiplication factor of 16, resulting in a system clock frequency of 64 MHz.

The AHB bus runs at the full system clock frequency (64 MHz), while the APB1 bus is prescaled by a factor of 2 to 32 MHz, and the APB2 bus operates at 64 MHz. This configuration satisfies the peripheral frequency constraints while maintaining maximum performance for time-critical components.

Particular attention is given to peripherals involved in motor control. The advanced timer TIM1 and the ADC are both clocked at 64 MHz, ensuring high timing resolution for PWM generation and precise synchronization of current sampling.

This clock distribution enables deterministic timing of control loops and supports the strict real-time requirements of the control algorithm.

TIM6 is clocked from the APB1 timer clock. Although the APB1 peripheral clock is prescaled to 32 MHz, the timer clock is doubled to 64 MHz due to the APB1 prescaler being different from one. TIM6 is configured with a prescaler of 31 and an auto-reload value of 999, resulting in an update interrupt frequency of 2 kHz.

3.3.2 Timer configuration

The advanced timer TIM1 is used for PWM generation. It is configured in center aligned mode, which reduces switching harmonics and is commonly used in motor control applications. The timer operates at a PWM frequency of 8888 Hz.

TIM1 also generates a compare event (`CC4`) that is used to trigger ADC injected conversions. This ensures that current sampling is synchronized with the PWM cycle and occurs at a well-defined

point in time.

A separate basic timer, TIM6, is used to periodically execute higher-level control loops. TIM6 operates at a frequency of 2 kHz and triggers an interrupt used for velocity and position control. This lower update rate reflects the cascaded structure of the control system, where slower outer loops are executed less frequently than the inner current control loop.

3.3.3 ADC configuration

The ADC is configured to perform injected conversions of three phase currents. The conversions are triggered by a TIM1 compare event, ensuring that current sampling occurs at a well-defined point within the PWM cycle. This synchronization minimizes measurement noise and improves control accuracy.

The ADC conversion complete interrupt is used to process the measured currents and execute the current control loop.

3.3.4 PWM ADC synchronization

Precise synchronization between PWM generation and current measurement is achieved by using a timer-generated trigger for ADC conversions. The TIM1 compare event (`CC4`) initiates the ADC injected sequence, and the corresponding interrupt is used to execute the control algorithm.

The trigger signal is generated by a dedicated compare channel that is not used for PWM output, allowing independent adjustment of the sampling point within the PWM cycle.

This approach ensures deterministic timing and minimizes control latency.

3.3.5 Communication interfaces

The MCU communicates with external systems primarily via the CAN interface, which is used for command exchange and system control. The CAN peripheral is configured in normal mode with automatic retransmission enabled.

The CAN peripheral is clocked from the APB1 bus operating at 32 MHz. The bit timing parameters are configured to achieve a communication speed of 500 kbit/s. This interface is used for command exchange and system control and operates independently of the real-time control loops.

A UART interface is also available and is used only for debugging and diagnostic output during development.

3.3.6 SPI interfaces

SPI communication is used to interface with external peripherals, namely the magnetic encoder and the gate driver. Both devices share a single SPI peripheral, with separate chip-select signals

used to access each device.

The SPI operates in master mode, and its configuration (clock polarity and phase) is adjusted as required by the connected peripherals.

The magnetic encoder is accessed within the real-time control loop to obtain rotor position information. The encoder update is performed inside the ADC interrupt service routine, ensuring that position data is synchronized with current measurements and control execution. As the SPI communication is performed in a blocking manner, its latency directly affects the execution time of the control loop.

In contrast, communication with the gate driver is primarily used for configuration. These operations are not time-critical and are performed outside the real-time control loop only during firmware initialization.

The SPI peripheral operates at a clock frequency of approximately 32 MHz, configured using a prescaler of 2. This high communication speed minimizes the duration of blocking SPI transactions, which is particularly important as encoder data acquisition is performed within the interrupt service routine.

At this speed, the encoder read transaction takes only a few microseconds.

3.4 Encoder

The rotor position is measured using an MT6835 magnetic encoder with a 21-bit absolute resolution and SPI communication interface. The encoder interface is implemented as a reusable module located in `uFOC_lib/encoder`. This module provides encoder initialization, raw position acquisition, CRC verification, position unwrapping, velocity estimation, signal filtering and electrical offset calibration.

The encoder is accessed through the SPI3 peripheral. Since the SPI bus is shared with the gate driver, the encoder module explicitly configures the SPI peripheral before encoder communication. The encoder uses a dedicated chip-select signal, while the SPI transfer itself is performed in blocking mode. This is acceptable because the encoder read operation is executed inside the real-time control loop and must therefore be deterministic and short.

3.4.1 Encoder data structure

The encoder state is stored in the `encoder_t` structure. It contains the most recent raw encoder value, the previous raw value, an unwrapped absolute position counter, velocity estimates, filtering parameters, electrical angle information and diagnostic variables related to CRC checking.

The raw encoder position is stored as a 21-bit value in the range from 0 to $2^{21} - 1$. To handle continuous rotation across the encoder overflow boundary, the module computes the difference between consecutive samples and corrects it when the difference exceeds half of the encoder range.

The corrected difference is accumulated into `position_ticks`, which represents the continuous mechanical position.

The structure also contains the `invert_dir` flag. This flag allows the logical direction of rotation to be inverted without modifying the raw encoder data. It affects the sign of the computed mechanical position, angular velocity and electrical angle.

Namely `encoder_t` contains:

- `bool invert_dir` – logical inversion of rotation direction which affects the sign of computed angle and velocity without modifying raw encoder data
- `uint32_t current_raw_value` – latest 21-bit raw position value read from the encoder via SPI
- `uint32_t prevous_raw_value` – previous raw position value, used to compute the delta between consecutive samples
- `float angular_velocity` – instantaneous angular velocity estimate (in rotations per second), computed from the filtered position increment
- `float angular_velocity_ewma` – exponentially weighted moving average (EWMA) of angular velocity which is used for velocity control to reduce signal noise
- `float angular_velocity_ewma_alpha` – EWMA coefficient α for velocity filtering, defines the trade-off between responsiveness and noise suppression
- `float ewma_value` – EWMA filtered absolute position value derived from `position_ticks`
- `float ewma_previous_value` – previous EWMA position value used for computing the filtered position difference
- `float ewma_delta` – difference between consecutive EWMA position values, serves as the basis for velocity estimation
- `float ewma_alpha` – EWMA coefficient α for position filtering, higher values reduce smoothing and improve responsiveness
- `int64_t position_ticks` – unwrapped absolute position in encoder ticks, accumulates deltas across revolutions without overflow
- `int32_t last_delta` – most recent position difference between samples
- `uint8_t valid` – data validity flag (currently not actively used)
- `uint8_t magnetic_pole_pairs` – number of motor pole pairs used to convert mechanical position to electrical angle
- `uint32_t electrical_offset` – calibrated offset between mechanical zero and electrical zero (obtained during calibration)

- `uint32_t electrical_angle_raw` – electrical angle expressed in raw encoder ticks (range 0 to 2^{21})
- `float electrical_angle` – electrical angle in radians which is used directly in FOC (Park/Clarke transforms)
- `uint32_t last_good_raw` – last valid raw encoder value (CRC verified), used as a fallback in case of communication errors
- `uint32_t crc_error_count` – counter of CRC errors detected during SPI communication with the encoder

3.4.2 Encoder initialization

The encoder is initialized using the `init_encoder()` function. During initialization, the first raw encoder value is read and used to initialize both the current and previous position values. The absolute position counter, EWMA filters and velocity variables are initialized from this first measurement.

The function also stores the number of motor pole pairs and the initial electrical offset. These parameters are required for conversion from mechanical position to electrical angle, which is used by the FOC algorithm.

3.4.3 Data reading and CRC control

Raw encoder data is read using the `mt6835_read_raw21()` function. The function sends a burst-read command to the MT6835 and receives six bytes over SPI. The 21-bit angle value is reconstructed from the received data bytes containing `ANGLE[20:13]`, `ANGLE[12:5]` and `ANGLE[4:0]`.

The encoder frame also contains status bits and an 8-bit CRC value. The module implements CRC-8 verification using a polynomial of `0x07` and an initial value of `0x00`. If the CRC check fails, the error counter `crc_error_count` is incremented and the last valid encoder value is reused. This prevents corrupted SPI measurements from directly affecting the control loop.

3.4.4 Position update and overflow handling

The encoder state is updated by calling `update_encoder()`. This function reads a new raw position value, verifies its CRC and computes the difference from the previous sample.

Because the encoder position is cyclic, the difference calculation must handle overflow at the end of the 21-bit range. If the measured difference is larger than half of the encoder modulo, the function assumes that the position crossed the overflow boundary and corrects the delta accordingly. The corrected delta is then accumulated into `position_ticks`.

This approach makes it possible to track continuous mechanical position over multiple revolutions while still using an absolute single-turn encoder.

3.4.5 Velocity calculation and signal filtering

The instantaneous angular velocity is calculated from the filtered position difference. The position signal is first processed by an exponentially weighted moving average filter. The filtered position difference is then converted from encoder ticks per sample to revolutions per second using the known sampling period.

The encoder module uses two EWMA filters. The first one filters the absolute position used for velocity estimation, while the second one filters the resulting angular velocity. This reduces measurement noise and improves the stability of the velocity control loop.

The velocity returned by `get_angular_velocity()` is the filtered angular velocity estimate. The direction inversion flag is also taken into account, so the returned value has the correct sign with respect to the configured motor direction.

3.4.6 EWMA filter

The exponentially weighted moving average (EWMA) is used in this implementation to filter noisy angle and angular velocity measurements. EWMA was chosen due to its low memory and computational requirements. Unlike a moving average, which requires storing `n` previous time series values, EWMA requires storing only one value. This value is then used to compute the new EWMA value and is subsequently overwritten. Therefore, EWMA represents a lightweight method for filtering signals on an embedded device with limited resources.

EWMA can be represented by the following equation:

$$EWMA_t = \alpha \cdot x_t + (1 - \alpha) \cdot EWMA_{t-1} \tag{3.1}$$

where $\alpha \in (0, 1)$ is a user-defined weighting parameter, x_t is the current value of the series, and $EWMA_{t-1}$ is the previous filtered value.

3.4.7 Electrical angle calculation

After the mechanical position is updated, the module calculates the electrical angle required by the FOC algorithm. First, the unwrapped mechanical position is reduced back into one mechanical revolution. This value is multiplied by the number of motor pole pairs to obtain the corresponding electrical position.

The calibrated electrical offset is then subtracted from this value. The result is wrapped into the encoder range and converted to radians:

$$\theta_e = \frac{2\pi \cdot \texttt{electrical\_angle\_raw}}{2^{21}} \tag{3.2}$$

The resulting electrical angle is stored in `electrical_angle` and is used directly by the Park and inverse Park transformations in the FOC control loop.

3.4.8 Electrical offset calibration

Accurate alignment between the mechanical position acquired by the encoder and the electrical angle required by the FOC algorithm is essential for correct motor operation. For this purpose, the encoder module provides the `calibrate_electrical_offset()` function.

This function is blocking and must only be executed when the motor is not operating under closed loop control. It is recommended to perform the calibration with the motor unloaded. In a typical workflow, the calibration should be executed only once, the resulting electrical offset is read via the CAN interface, stored and applied during normal operation either by client side updating the electrical offset via can interface or recompiling the firmware with adjusted default `ENCODER_ELECTRICAL_OFFSET` in `config.h` without repeating the calibration during firmware initialization.

At the start of the calibration procedure, the ADC injected conversion interrupt is disabled, to stop the current control loop. A static voltage vector is then applied to the motor using the inverse Park and inverse Clarke transformations followed by SVPWM duty cycle computation. This forces the rotor into settle in a known and stable electrical position.

After a short settling period, a sequence of encoder samples is collected and averaged in order to reduce measurement noise. This value is then converted into the corresponding electrical position by accounting for the number of motor pole pairs:

$$\theta_{e\_raw} = \left(\theta_{m\_raw} \cdot p\right) \bmod 2^{21} \tag{3.3}$$

where θ_{m_raw} represents averaged mechanical position in encoder ticks and p represents number of motor pole pairs.

The computed value θ_{e_raw} in a known mechanical position is then used as the electrical offset θ_{offset} between the encoder mechanical zero and the actual electrical zero of the motor.

This offset is then used during normal operation to compute the electrical angle in radians as:

$$\theta_e = \frac{2\pi}{2^{21}} \left(\theta_{m\_raw} \cdot p - \theta_{\text{offset}}\right) \tag{3.4}$$

and is a crucial input to the FOC algorithm.

Once the calibration procedure is complete, the PWM outputs are returned to a neutral duty cycle and the ADC interrupt is re-enabled, restoring the standard control loop execution. The calibrated offset is then consistently applied during runtime, ensuring correct phase alignment required for stable FOC operation.

3.5 BLDC motor driver

To control and measure currents in motor windings, a DRV8316C three-phase BLDC motor driver with built-in current sensing is used. The driver provides a configurable 6x or 3x PWM control scheme. In this implementation, the 6 PWM mode is used, which allows full control over both high-side and low-side transistors, enabling the implementation of advanced modulation strategies such as SVPWM and optimized current sensing.

Communication with the gate driver is implemented via an SPI interface, which is shared with the magnetic encoder. Since both peripherals require different SPI configurations (clock polarity and phase), the SPI peripheral is reconfigured before each transaction depending on the selected device. Each device is accessed using a dedicated chip-select signal, ensuring that only one peripheral is active on the bus at a time.

Unlike communication with the magnetic encoder, communication with the motor driver is not time-critical and is performed only during firmware initialization. As a result, it does not interfere with the time-critical execution of the FOC control loop.

The DRV8316C interface is implemented as a reusable module located in `uFOC_lib/driver`. This module encapsulates all low-level interactions with the motor driver, including SPI communication and device configuration. It also provides an abstraction for current sensing and PWM generation, allowing the higher-level control logic to remain independent of the specific hardware implementation. By isolating driver-specific functionality into a dedicated module, the firmware maintains a modular structure and can be more easily adapted to different motor drivers if required.

3.5.1 PWM generation

PWM generation for the three motor phases is handled using the advanced timer TIM1. Since the DRV8316C driver is operated in 6 PWM mode, three main PWM outputs and three complementary PWM outputs are used to control the high-side and low-side transistors of the three-phase bridge.

The PWM outputs are started by the `pwm_init()` function, which enables all three PWM channels together with their complementary outputs. The duty cycles are updated using the `pwm_set()` function. This function receives normalized duty cycle values for phases A, B and C in the range from 0 to 1. Before writing the values to the timer compare registers, the duty cycles are limited to this valid range in order to prevent invalid PWM states.

The normalized duty cycle values are converted to timer compare values according to the auto-reload value of TIM1. The resulting compare values are then written to the corresponding timer channels, which directly determine the switching duty cycle of each motor phase. This abstraction allows the FOC algorithm to work with normalized phase voltage commands without directly accessing timer registers.

In normal operation, the duty cycle values are produced by the SVPWM algorithm and passed to the driver module as normalized phase commands.

3.5.2 SPI communication and driver configuration

The DRV8316C is configured through the SPI interface. Since the SPI peripheral is shared with the magnetic encoder, the driver module first configures SPI3 to match the communication mode required by the gate driver before each transaction. For the DRV8316C, SPI is configured with low clock polarity and data sampling on the second clock edge.

Communication with the driver is implemented using 16-bit SPI frames. Each frame contains a read/write bit, a 6-bit register address, an 8-bit data field and a parity bit. The parity bit is calculated from the remaining frame bits before transmission and is used by the driver to detect communication errors.

The driver initialization is performed by the `drv8316_init()` function. At the beginning of the initialization sequence, the driver chip-select signal is disabled followed by a short delay to ensure a stable idle state of the SPI bus. The driver registers are then unlocked and the required configuration values are written to the control registers.

The `CTRL2` register is configured to clear existing fault flags, select the 6 PWM control mode, set the output slew rate to 50 V/μs and enable the SDO mode required for SPI readback. The `CTRL5` register is used to configure the gain of the integrated current sense amplifiers according to the selected `drv8316_csa_gain_t` value.

This value is passed as an argument to the `drv8316_init()` function and should be selected by the user based on the used motor and expected current draw. An enum structure `drv8316_csa_gain_t` was implemented to create an abstraction layer for the `drv8316_csa_gain_t` value and is later used for correct current conversion.

The `CTRL6` register configures the internal buck converter for a 3.3 V output. Setting the buck converter for a 3.3 V output is critical. Higher voltage will likely result in an instant MCU demidge.

3.5.3 Current sensing

Phase current measurement is implemented using the integrated current sense amplifiers (CSA) available in the DRV8316C motor driver.

Current sensing is configured during the driver initialization sequence through the `CTRL5` register, where the gain of the internal current sense amplifiers is selected using the `drv8316_csa_gain_t` enumeration. The selected gain directly affects the measurable current range and the current measurement resolution. Lower gain values allow measurement of higher currents, while higher gain values provide better resolution for lower current levels. The avialable gain configurations correspond to current conversion factors of 0.15, 0.3, 0.6 and 1.2 V/A.

The amplified current sense signals are connected to ADC inputs of the STM32 microcontroller. Current sampling is synchronized with the PWM timer in order to obtain measurements at deterministic points within the PWM cycle and minimize the influence of switching noise. The ADC conversions are triggered directly by the timer hardware, ensuring stable sampling timing independent of software execution latency.

The PWM timer operates in center-aligned mode, which ensures symmetrical switching of the inverter transistors during the PWM period. This creates a stable interval around the middle of the PWM cycle where the phase currents are minimally affected by switching transients. As a result, ADC sampling performed in this interval provides more accurate and repeatable current measurements.

The measured ADC values are converted to phase currents using the known ADC reference voltage, ADC resolution, shunt resistor value, and the configured CSA gain. Since the current sense amplifier outputs are offset to a reference voltage representing zero current, the measured ADC value must first be shifted by a calibrated offset value before the phase current is calculated.

The driver implements the `driver_current_offsets_t` structure to store calibrated offsets for each ADC channel separately. Since the ADC operates with 12-bit resolution, the `uint16_t` data type is used to store the offset values. Furthermore, the `driver_phase_currents_t` structure is implemented to store the latest current measurements for individual motor phases as floating-point values in amperes.

During normal operation of the FOC current control loop, the `driver_phase_currents_from_adc()` function is used to convert raw ADC measurements into phase currents. The function receives raw ADC samples together with the calibrated offsets and configured CSA gain, and stores the converted current values into the `driver_phase_currents_t` structure.

3.5.4 Current sensing offset calibration

The current sense amplifiers generate an output voltage centered around a predefined zero-current reference level, approximately in the middle of the ADC input range. Therefore, an offset calibration procedure is performed during firmware initialization.

The driver module implements the `driver_current_calibrate_offsets()` function, which receives a pointer to the `driver_current_offsets_t` structure, pointers to variables containing raw ADC current samples, the number of calibration samples, and the delay between samples in milliseconds.

The function itself does not perform ADC sampling directly. Therefore, the raw ADC current values must be continuously updated by the TIM1 interrupt routine while the calibration procedure is running. It is also required that the PWM outputs are already initialized and all PWM channels are set to a duty cycle of 0.5, ensuring that no phase current flows through the motor windings during calibration. Failure to satisfy these conditions may result in incorrect

offset calibration.

The calibration function averages the acquired ADC samples and stores the calculated offset values into the `driver_current_offsets_t` structure.

3.6 FOC implementation

3.6.1 Park Clarke

3.6.2 Trigonometric functions optimization

3.6.3 Current control loop

3.6.4 Velocity and position control loops

3.7 Configuration abstraction layer

4 | Conclusion

4.1 Future work

Appendices

A | Supplementary figures

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Aliquam erat volutpat. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Vivamus porttitor turpis ac leo. Aliquam ornare wisi eu metus. Aliquam ante. Nulla quis diam. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Duis ante orci, molestie vitae vehicula venenatis, tincidunt ac pede. Curabitur bibendum justo non orci. Duis sapien nunc, commodo et, interdum suscipit, sollicitudin et, dolor. Phasellus et lorem id felis nonummy placerat.

Donec vitae arcu. Vivamus luctus egestas leo. Aliquam erat volutpat. Mauris dictum facilisis augue. Integer vulputate sem a nibh rutrum consequat. Proin mattis lacinia justo. Aliquam id dolor. In enim a arcu imperdiet malesuada. Duis sapien nunc, commodo et, interdum suscipit, sollicitudin et, dolor. Nunc dapibus tortor vel mi dapibus sollicitudin. Duis pulvinar. Praesent vitae arcu tempor neque lacinia pretium. Nam libero tempore, cum soluta nobis est eligendi optio cumque nihil impedit quo minus id quod maxime placeat facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Nulla quis diam.

B | Advance things

Aliquam erat volutpat. Duis risus. Fusce suscipit libero eget elit. Suspendisse sagittis ultrices augue. Nullam feugiat, turpis at pulvinar vulputate, erat libero tristique tellus, nec bibendum odio risus sit amet ante. In convallis. Fusce wisi. Nulla quis diam. Sed elit dui, pellentesque a, faucibus vel, interdum nec, diam. Pellentesque arcu. Aliquam ornare wisi eu metus. Nullam sit amet magna in magna gravida vehicula. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet ut et voluptates repudiandae sint et molestiae non recusandae. Mauris suscipit, ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis velit mauris vel metus.

References